

Anatomie d'un Agent IA

Les parties qu'on retrouve dans presque tout script d'agent IA en Python, et pourquoi chacune existe.

1. Imports / dependances

```
import os
from dotenv import load_dotenv
from pydantic import BaseModel
from agno.agent import Agent
```

POURQUOI

On ne reinvente rien. Chaque bibliotheque resout un probleme deja resolu ailleurs (lire un .env, valider des donnees, parler a un LLM). Sans elles, il faudrait reecrire soi-meme la gestion des cle API, la validation de types, les appels HTTP aux fournisseurs de LLM... Le code se concentre sur la LOGIQUE METIER, pas sur la plomberie.

2. Configuration / variables d'environnement

```
load_dotenv()
API_KEY = os.getenv("MA_CLE_API")
```

POURQUOI

Separer le CODE (qui ne change jamais) des SECRETS/PARAMETRES (qui changent selon la personne, l'environnement, le compte). Ca evite de coder une cle API en dur dans un fichier qui pourrait finir sur GitHub, et permet a chacun d'utiliser ses propres identifiants sans toucher au code.

3. Client d'API externe

```
client = ServiceExterne(api_key=API_KEY)
```

POURQUOI

Une seule instance du client, creee une fois, reutilisee partout dans le fichier -- plutot que de la recreer a chaque appel de fonction (gaspillage de ressources, et parfois un objet client gere une connexion/session qu'on veut garder ouverte).

4. Modeles de donnees (Pydantic, dataclasses...)

```
class Resultat(BaseModel):
    titre: str
    valeur: int
```

POURQUOI

Donner une FORME FIXE ET VERIFIEE aux donnees qui circulent dans le programme. Sans ca, on manipule des dictionnaires generiques (dict) ou une cle manquante ou mal orthographiee ne se revele qu'au moment ou le programme plante, potentiellement loin de l'endroit ou l'erreur a ete commise. Avec un modele, l'erreur est detectee immediatement, a la creation de l'objet.

5. Definition de l'agent / du LLM

```
agent = Agent(  
    model=Modele(id="..."),  
    instructions="...",  
    output_schema=Resultat, # optionnel  
)
```

POURQUOI

Centraliser en un seul endroit QUEL modele on utilise, COMMENT il doit se comporter (ses instructions/son role), et SOUS QUELLE FORME il doit repondre. C'est la configuration du comportement de l'IA, separee du code qui l'appelle -- on peut changer les instructions sans toucher au reste du programme.

6. Fonctions de recuperation (retrieval)

```
def recuperer_donnees(parametre):  
    resultat_brut = client.appel_quelconque(parametre)  
    return nettoyer(resultat_brut)
```

POURQUOI

Isoler l'appel a un service externe (API, base de donnees, fichier...) dans sa propre fonction. Ca rend le code testable independamment (on peut verifier que la recuperation marche AVANT de brancher le LLM dessus), et ca isole les echecs possibles (site bloque, timeout, format inattendu) a un seul endroit qu'on peut gerer proprement (retourner None, une liste vide...) plutot que de laisser une erreur remonter n'importe ou.

7. Fonction d'orchestration

```
def traiter(entree):  
    donnees = recuperer_donnees(entree)  
    if not donnees:  
        return "Rien trouve"  
    return agent.run(construire_prompt(donnees)).content
```

POURQUOI

C'est le "chef d'orchestre" du programme -- la seule fonction qui connait L'ORDRE DES ETAPES a executer (d'abord recuperer, puis construire le prompt, puis appeler le LLM, puis renvoyer le resultat). Toutes les autres fonctions ne savent faire qu'une seule chose ; c'est celle-ci qui les enchaîne dans le bon ordre.

CONCRETEMENT, ELLE FAIT 3 CHOSES

1. **Elle appelle les autres fonctions dans le bon ordre.** Sans elle, il faudrait repeter cette sequence (recuperer -> verifier -> construire le prompt -> appeler le LLM) a chaque endroit du programme qui en a besoin -- un copier-coller de logique, source classique de bugs quand on oublie de mettre a jour une des copies.
2. **Elle gere le cas ou la recuperation echoue, AVANT d'appeler le LLM.** Le "if not donnees: return" evite un appel LLM inutile (et couteux) si on sait deja qu'on n'a rien a lui donner a traiter.
3. **C'est le seul point d'entree dont le reste du programme a besoin.** L'interface (ou un autre script, ou un test) l'appelle avec une entree, recupere une sortie -- tout ce qui se passe a l'interieur (combien de fonctions, dans quel ordre, avec quelle logique) est un detail cache, qu'on peut changer sans casser ce qui l'appelle.

L'idée derriere : separer le "COMMENT ON RECUPERE" (fonction de recuperation), le "COMMENT ON DECIDE/REDIGE" (l'agent LLM), et le "DANS QUEL ORDRE ON FAIT TOUT CA" (la fonction d'orchestration). Si demain la facon de recuperer les donnees change completement, seule la fonction de recuperation change -- l'orchestration et l'agent restent identiques, parce qu'ils ne connaissent jamais les details internes les uns des autres, juste ce qu'ils recoivent et ce qu'ils doivent renvoyer.

8. Interface (Streamlit, API web, CLI...)

```
if st.button("Lancer"):
    resultat = traiter(entree_utilisateur)
    st.markdown(resultat)
```

POURQUOI

Decoupler la LOGIQUE (comment ca marche) de la PRESENTATION (comment on l'utilise). Ca permet de brancher n'importe quelle interface (web, ligne de commande, API REST) sur la meme logique metier sans la dupliquer.

Le principe general derriere tout ca : la separation des responsabilites. Chaque partie fait UNE seule chose (config, donnees, recuperation, decision, presentation), ce qui rend le code plus facile a lire, tester, deboguer et faire evoluer -- un bug dans l'affichage n'a aucune chance de venir de la logique de recuperation, et inversement.